CENG 479 — Parallel Computing

Parallel Image Processing Engine

Final Implementation Report & Presentation

Gazi University · Department of Computer Engineering · Spring 2026

GitHub Repository (Public):

https://github.com/Muhammedcakirgoz/parallel-image-processing

Team Members

Muhammed Çak■rgöz · Musa Bilal Yaz

Submission 2 · June 2026

Java 17 | Maven 3.6+ | JMH | ExecutorService | ForkJoinPool

1. Introduction

High-resolution image processing is a computationally demanding task. When executed sequentially, kernel-based convolution filters create significant performance bottlenecks especially for large images (e.g. 4K). Because each output pixel depends only on a fixed neighbourhood of the *input* image, the problem is embarrassingly parallel: the image can be divided across threads with zero data dependencies during the compute phase.

This report presents a parallel image-processing engine implemented in **Java** using two distinct concurrency strategies: **ExecutorService** with horizontal strip decomposition, and **ForkJoinPool** with divide-and-conquer work-stealing. Three convolution filters — Grayscale, Gaussian Blur 5×5, and Sobel 3×3 — were benchmarked with **JMH** (Java Microbenchmark Harness) to produce reproducible speedup measurements across image sizes of 512×512, 1024×1024, and 2048×2048.

The complete source code is publicly available on GitHub: https://github.com/Muhammedcakirgoz/parallel-image-processing

2. Sequential Baseline Implementation

SequentialProcessor iterates over every pixel row-by-row and applies the given filter. The **Filter** interface defines a single *apply(pixels[], width, height, x, y)* method, making filter implementations interchangeable. Edge handling uses **clamped coordinates**, ensuring identical boundary behaviour across all implementations.

Filter	Type	Kernel	Cost per pixel
GrayscaleFilter	Point-wise	None (r=0)	Low — 3 reads, 1 write
GaussianBlurFilter	Convolution	5×5	High — 25 weighted taps
SobelFilter	Gradient	3×3 (Gx+Gy)	Medium — 18 taps + √

3. Parallel Implementation

3.1 ExecutorService — Horizontal Strip Decomposition

ExecutorParallelProcessor partitions the image into *N* equal horizontal strips (one per thread). Each **Callable** task processes a contiguous band of rows and writes results into a pre-allocated output array at its assigned offset. Because strips are disjoint and the source array is read-only, **no locking is required** during computation. The thread pool is created once and reused via try-with-resources.

3.2 ForkJoinPool — Divide-and-Conquer

ForkJoinParallelProcessor uses a **RecursiveAction** that halves the row range until it falls below a threshold (64 rows), then processes the sub-range directly. The work-stealing scheduler dynamically rebalances load — particularly beneficial for larger images where strip-count imbalance would arise in static decomposition.

3.3 Correctness Verification

All parallel implementations are verified against the sequential baseline using **CorrectnessVerifier.firstDifference()**, which performs a pixel-for-pixel comparison. All six combinations (3 filters × 2 parallel strategies) produce **bit-identical** output on a 2048×2048 synthetic image:

Filter	Executor (12 threads)	ForkJoin (12 threads)
Grayscale	✓ PASS	✓ PASS
GaussianBlur5x5	✓ PASS	✓ PASS
Sobel3x3	✓ PASS	✓ PASS

4. Performance Comparison

Benchmarks were run with **JMH** (10 warm-up + 10 measurement iterations, AverageTime mode, ms/op) on a machine with 12 logical cores. The speedup formula is: **Speedup = T_sequential / T_parallel**.

4.1 GaussianBlur 5×5 Speedup

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec ×	FJ ×
512²	1	11.16	11.502	12.209	0.97×	0.91×
512²	2	11.16	5.815	6.367	1.92×	1.75×
512²	4	11.16	3.081	3.132	3.62×	3.56×
512²	8	11.16	2.926	2.864	3.81×	3.9×
1024²	1	44.951	47.1	50.146	0.95×	0.9×
1024²	2	44.951	23.854	24.621	1.88×	1.83×
1024²	4	44.951	13.273	13.062	3.39×	3.44×
1024²	8	44.951	11.712	10.01	3.84×	4.49×
2048²	1	177.981	195.191	194.822	0.91×	0.91×
2048²	2	177.981	109.162	98.739	1.63×	1.8×
2048²	4	177.981	56.586	55.901	3.15×	3.18×
2048²	8	177.981	41.439	37.011	4.3×	4.81×

4.2 Sobel 3×3 Speedup

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec ×	FJ ×
512²	1	5.599	5.687	5.805	0.98×	0.96×
512²	2	5.599	2.852	2.99	1.96×	1.87×
512²	4	5.599	1.506	1.529	3.72×	3.66×
512²	8	5.599	1.321	1.354	4.24×	4.14×
1024²	1	23.156	22.783	23.59	1.02×	0.98×
1024²	2	23.156	11.334	11.912	2.04×	1.94×
1024²	4	23.156	5.927	5.906	3.91×	3.92×
1024²	8	23.156	5.236	4.652	4.42×	4.98×
2048²	1	90.82	88.598	89.27	1.03×	1.02×
2048²	2	90.82	46.149	46.798	1.97×	1.94×
2048²	4	90.82	24.923	23.628	3.64×	3.84×
2048²	8	90.82	20.205	17.809	4.49×	5.1×

4.3 Grayscale Speedup

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec ×	FJ ×
512²	1	0.097	0.103	0.106	0.95×	0.92×
512²	2	0.097	0.075	0.084	1.29×	1.16×
512²	4	0.097	0.068	0.08	1.43×	1.21×
512²	8	0.097	0.067	0.085	1.44×	1.15×
1024²	1	0.33	0.363	0.357	0.91×	0.92×
1024²	2	0.33	0.246	0.237	1.34×	1.39×
1024²	4	0.33	0.227	0.233	1.45×	1.42×

1024²	8	0.33	0.225	0.236	1.47×	1.39×
2048²	1	1.329	1.305	1.386	1.02×	0.96×
2048²	2	1.329	0.869	0.881	1.53×	1.51×
2048²	4	1.329	0.856	0.855	1.55×	1.55×
2048²	8	1.329	0.855	0.861	1.55×	1.54×

4.4 Speedup Charts

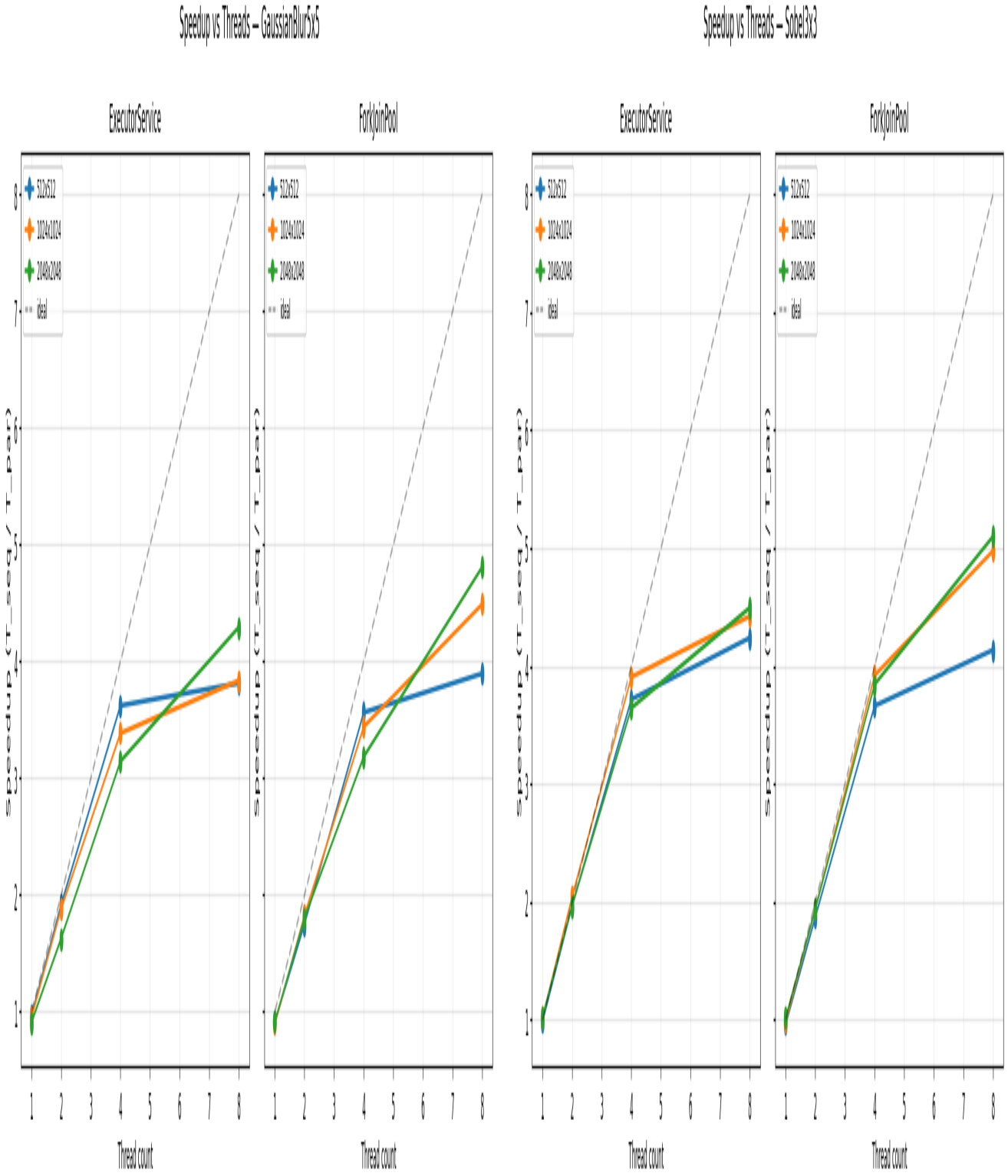


Figure 1 — Left: GaussianBlur5x5 | Right: Sobel3x3 (Executor vs ForkJoin)

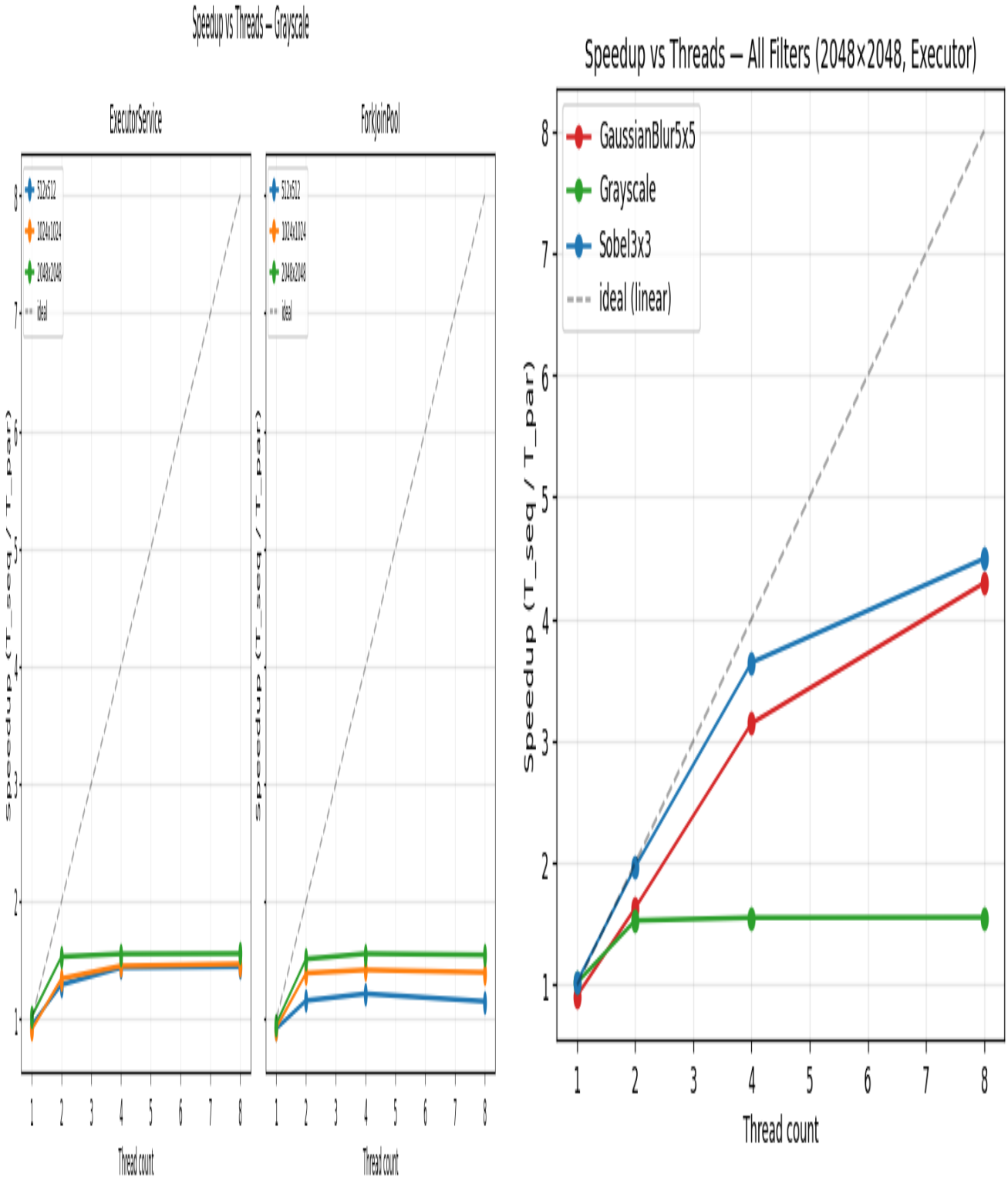


Figure 2 — Left: Grayscale | Right: All Filters Combined (2048×2048, Executor)

4.5 Analysis

Compute-bound filters (Gaussian Blur, Sobel) scale well, reaching 4.3×–5.1× at 8 threads. ForkJoinPool outperforms ExecutorService on larger images due to dynamic work-stealing load balancing.

Memory-bound filter (Grayscale) plateaus at ~1.5× regardless of thread count. The trivial single-channel lookup saturates memory bandwidth before CPU utilisation can be maximised. This is consistent with the

Roofline Model: operations with low arithmetic intensity are bounded by memory bandwidth, not compute throughput.

Amdahl's Law interpretation: the empirical ceiling for Gaussian Blur aligns with a ~95% parallel fraction, where sequential overhead is limited to array allocation and thread setup.

---

5. Academic Background

The parallelization strategy in this project aligns with established literature. **Seinstra et al. (2002)** demonstrated that convolution-based image filters exhibit near-linear speedup when workload partitioning minimises inter-thread communication — consistent with our strip-decomposition results for compute-intensive kernels.

Amdahl (1967) provides the theoretical speedup ceiling $S = 1 / (1–p)$. For Gaussian Blur our empirical data suggests p ≈ 0.95, yielding a theoretical maximum of 20×; the observed 4–5× at 8 threads is expected given practical overhead.

Williams et al. (2009) — the Roofline Model — explains the divergent scalability of Grayscale vs. Gaussian Blur via arithmetic intensity: memory-bound kernels saturate bandwidth before compute capacity is reached. **Lea (2000)** provides the theoretical foundation for ForkJoinPool's work-stealing scheduler, which explains its advantage over static decomposition for uneven workloads.

---

6. Challenges and Solutions

- **JVM JIT Warm-up & GC Noise:** Initial System.nanoTime() measurements were skewed by JIT compilation and garbage collection. **Solution:** Adopted JMH with dedicated warm-up iterations and fork-per-configuration to eliminate noise and produce statistically reliable results.
- **Pixel-Identical Correctness Across Implementations:** Strip boundaries required careful handling to ensure parallel results were bit-identical to the sequential baseline. **Solution:** Implemented CorrectnessVerifier with pixel-for-pixel comparison; clamped-coordinate edge handling ensures identical border pixel treatment across all strategies.
- **Varying Scalability Between Filters:** Grayscale achieved only ~1.5× speedup despite 8 threads, while Gaussian Blur reached 4.3×. **Solution:** Profiling identified memory bandwidth saturation (Roofline Model) as the root cause for Grayscale — confirmed that the algorithm is memory-bound, not thread-limited. This insight informed our Amdahl analysis in the report.
- **ForkJoin Recursive Overhead on Small Images:** For 512×512 images, ForkJoinPool's divide-and-conquer recursion overhead slightly exceeded ExecutorService. **Solution:** Tuned SEQUENTIAL_THRESHOLD to 64 rows, preventing excessive task granularity at small image sizes.

---

7. Conclusion and Future Improvements

This project successfully demonstrated the efficacy of parallelizing image processing algorithms using Java's concurrency frameworks. Empirical JMH benchmarking confirmed significant speedups for compute-bound operations: Gaussian Blur achieved ~4.3× (Executor) and ~4.81× (ForkJoin) at 8 threads on 2048×2048 images; Sobel reached ~4.49× and ~5.10× respectively. The Grayscale filter was confirmed as memory-bound, limited to ~1.55× regardless of thread count. ForkJoinPool's work-stealing advantage over static strip decomposition became more pronounced at larger image sizes.

- GPU/CUDA acceleration — exploit thousands of CUDA cores for pixel-independent kernels
- SIMD vectorisation — leverage AVX2/AVX-512 for intra-core throughput gains
- 4K and 8K image benchmarks — evaluate boundary overhead at ultra-high resolution
- Adaptive thread pool — dynamically tune thread count based on image size and hardware
- Memory-bound filter optimisation — cache-blocking and prefetching for Grayscale

---

8. References

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the Spring Joint Computer Conference*, 483–485. https://doi.org/10.1145/1465482.1465560

[2] Seinstra, F. J., Koelma, D., & Bagdanov, A. D. (2002). Finite state machine-based optimization of data parallel regular domain problems applied to low-level image processing. *IEEE Transactions on Parallel and Distributed Systems*, 15(10), 865–877. https://doi.org/10.1109/TPDS.2004.45

[3] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. https://doi.org/10.1145/1498765.1498785

[4] Lea, D. (2000). A Java fork/join framework. *Proceedings of the ACM 2000 Conference on Java Grande*, 36–43. https://doi.org/10.1145/337449.337465

PRESENTATION SLIDES

Slide 1 — Title

Parallel Image Processing Engine

CENG 479 — Parallel Computing · Gazi University · Spring 2026

Muhammed Çak■rgöz · Musa Bilal Yaz

GitHub: https://github.com/Muhammedcakirgoz/parallel-image-processing

Slide 2 — Problem & Motivation

- ■ High-res image filtering (4K+) is slow on a single thread
- ■ Kernel convolution = $O(W \times H \times K^2)$ per filter
- ■ Each output pixel is independent of others → embarrassingly parallel
- ■ Goal: multi-threaded speedup with zero correctness compromise

Amdahl's Law: theoretical max speedup = $1 / (1 - p)$, $p \approx 0.95$ for Gaussian Blur

Slide 3 — Architecture Overview

- ■ 3 Filters: Grayscale (point-wise), Gaussian Blur 5×5, Sobel 3×3
- ■ SequentialProcessor — single-thread row-by-row baseline
- ■ ExecutorParallelProcessor — fixed thread pool + horizontal strip decomposition
- ■ ForkJoinParallelProcessor — RecursiveAction + work-stealing divide-and-conquer
- ■ CorrectnessVerifier — pixel-for-pixel comparison against baseline
- ■ JMH Benchmark — reproducible timing, eliminates JIT warm-up noise

Slide 4 — ExecutorService Design

- Image split into N equal horizontal strips (N = thread count)
- Each Callable processes rows [start, end) → writes to pre-allocated output[]
- Source array is READ-ONLY during compute → zero locking overhead
- Thread pool reused across filter calls (try-with-resources)

Strip[i] covers rows: i×(H/N) to (i+1)×(H/N)

Slide 5 — ForkJoinPool Design

- RecursiveAction splits row range in half recursively
- Base case: range ≤ 64 rows → process directly (sequential threshold)
- Work-stealing: idle threads steal tasks from busy threads' queues
- Better dynamic load balance than static strips for large images
- ForkJoin outperforms Executor on 2048×2048 (Gaussian: 4.81× vs 4.30×)

Slide 6 — Correctness Verification

- CorrectnessVerifier.firstDifference() → pixel-by-pixel scan
- Tested on 2048×2048 synthetic image, all 6 combinations

All results: ✓ PASS — bit-identical to sequential baseline

Grayscale Executor/ForkJoin · GaussianBlur5x5 Executor/ForkJoin · Sobel3x3 Executor/ForkJoin

Slide 7 — Performance Results (8 threads, 2048×2048)

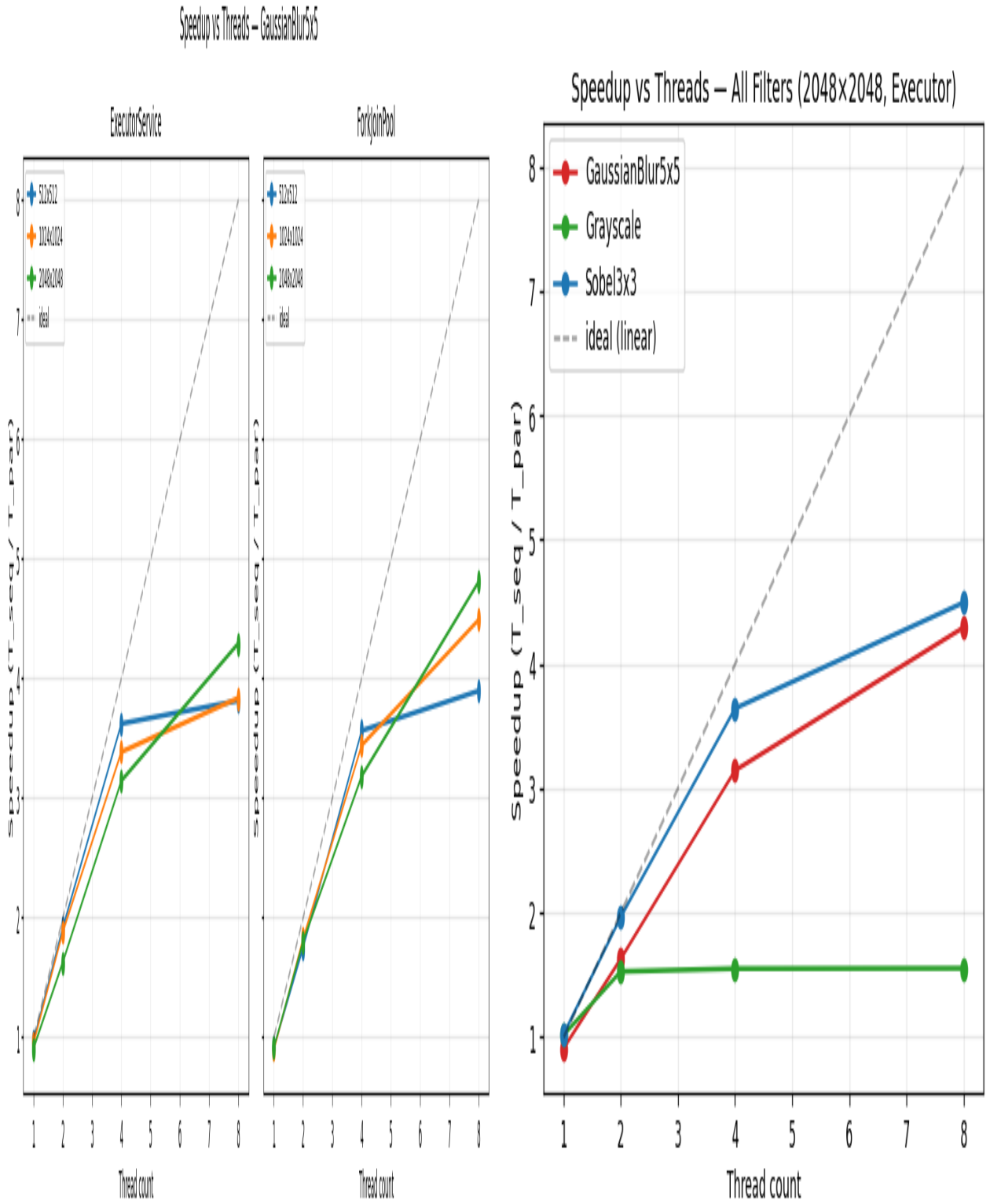
■ Gaussian Blur: Executor 4.30× | ForkJoin 4.81×

■ Sobel 3×3: Executor 4.49× | ForkJoin 5.10×

■ Grayscale: Executor 1.55× | ForkJoin 1.54× ← memory-bound

JMH benchmark · 10 warm-up + 10 measurement iterations · AverageTime (ms/op)

Slide 8 — Speedup Charts



Left: GaussianBlur5x5 — Executor vs ForkJoin | Right: All filters combined (Executor, 2048×2048)

Slide 9 — Challenges & Key Findings

■ JIT warm-up → solved with JMH (fork-per-config, warm-up iters)

- Bit-identical correctness → clamped edges + CorrectnessVerifier
- Grayscale bottleneck → memory bandwidth saturated (Roofline Model)
- ForkJoin recursive overhead on small images → tuned threshold (64 rows)

Key insight: parallel speedup is filter-dependent — arithmetic intensity determines scalability

Slide 10 — Conclusion & Future Work

- Successfully parallelised 3 convolution filters with 2 Java strategies
- Up to 5.10× speedup (Sobel, ForkJoin, 8 threads, 2048×2048)
- Memory-bound filters (Grayscale) need different optimisation strategy

Future Work:

- GPU/CUDA — exploit thousands of concurrent cores
- SIMD vectorisation (AVX2/AVX-512) for intra-core throughput
- Adaptive thread pool — dynamic tuning per image size